BUILDABLE BLUEPRINT · V1.0 · 01 AUGUST 2026

State, Structure, Shipping

The exact data domains and repository boundaries that turn the approved YT Growth Stack architecture into safe, reviewable work.

Database
Supabase Postgres + Auth + Storage + Realtime

Repository
Modular pnpm workspace with one Next.js application

Operating rule
Tenant data is explicit. Provider internals stay server-only.

SECURITY ANCHOR
Workspace is the boundary

auth user -> membership -> workspace -> every customer-owned record

No workspace membership = no row, file, event, or realtime channel

DOMAIN 01

Identity

Who the customer is and which workspace roles they hold.

profiles
product user linked to auth.users

workspaces
tenant and plan owner

workspace_members
owner, admin, editor, viewer

workspace_settings
retention, voice, brand, locale

DOMAIN 02

Connections

Owned and competitor channels plus protected provider access.

provider_connections
status, scopes, expiry metadata

provider_secrets
private encrypted secret reference

channels
owned or competitor channel

channel_snapshots
channel metrics over time

videos
normalized video metadata

video_snapshots
video metrics over time

DOMAIN 03

Conversation

Voice and chat form one durable, reviewable customer thread.

conversations
agent work thread

messages
user, agent, tool, system

voice_sessions
model, duration, latency, usage

transcripts
editable versioned voice text

DOMAIN 04

Research

Durable jobs collect and normalize traceable evidence.

research_projects
request and approved scope

research_targets
channel, niche, keyword, URL

jobs + job_events
state and progress history

provider_runs
one external invocation

source_documents
normalized source object

evidence_items
claim-bearing source fragment

DOMAIN 05

Content

Ideas become versioned, approved creative artifacts.

ideas + idea_scores
candidate and scoring versions

content_packages
selected idea workspace

artifacts
title, thumbnail, hook, outline, script

artifact_versions
immutable generated or edited version

approval_requests
pending human decision

approval_events
append-only decision history

DOMAIN 06

Operations

Models, costs, billing, callbacks, and audits stay accountable.

prompt_versions
prompt and output schema version

model_runs
model, tokens, latency, cost

usage_events
append-only metering ledger

subscriptions
billing provider mirror

webhook_receipts
private deduplication inbox

audit_events
security and mutation trail

One workspace
owns channels, conversations, research, content, usage, and approvals

One job
has many events and provider runs; state is durable in Postgres

One source
has many evidence items; every claim keeps provenance

One artifact
has immutable versions; approval names the exact version reviewed

SUPABASE CONTROL PLANE

Security is enforced where the data lives

Server checks improve the experience; Postgres RLS remains the final tenant boundary. Credentials and webhook internals do not enter the browser-facing schema.

Request authorization path

EVERY TENANT QUERY

01
Authenticated user
Supabase session provides `auth.uid()`.

>

02
Membership check
Indexed helper confirms workspace and allowed role.

>

03
Row or private channel
Policy permits only the matching workspace boundary.

Outsider, anonymous caller, guessed UUID, or changed workspace ID: denied.

Role matrix

MVP

ACTION	OWNER	ADMIN	EDITOR	VIEWER
Read workspace content	Yes	Yes	Yes	Yes
Create research/content	Yes	Yes	Yes	No
Approve content	Yes	Yes	Yes	No
Manage connections	Yes	Yes	No	No
Manage members	Yes	Limited	No	No
Billing / delete workspace	Yes	No	No	No

PRIVATE BUCKET 01

raw-research
Raw Apify, Firecrawl, YouTube, and model exports. Service jobs write; support tooling reads.

`workspace/project/provider/object`

PRIVATE BUCKET 02

workspace-assets
Customer uploads, approved reference images, and thumbnail assets. Editors write; members read.

`workspace/project/assets/object`

PRIVATE BUCKET 03

exports
Generated briefs, scripts, and package exports. Members receive time-limited signed URLs.

`workspace/package/version/export`

Durable job state

REALTIME IMPROVES UX; POSTGRES IS TRUTH

01	02	03	04	05	06	07	08
queued	collecting	normalizing	analysing	awaiting approval	generating	validating	completed

Verification: pgTAP tests every RLS policy with owner, editor, viewer, outsider, anonymous, and service contexts. Provider callback fixtures are replayed twice to prove idempotency. Database and Storage recovery are tested separately.

AGENT-READABLE REPOSITORY

Put context where the work happens

A small root contract applies everywhere. Narrower instruction files and focused skills appear only where database, web, or provider work has different safety rules.

```
YT Growth Stack/
|-- AGENTS.md
|-- README.md
|-- package.json
|-- pnpm-workspace.yaml
|-- turbo.json
|-- .env.example
|-- .agents/skills/
|   |-- add-feature-slice/
|   |-- add-supabase-migration/
|   |-- add-provider-adapter/
|   |-- change-ai-prompt/
|   |-- diagnose-job-failure/
|   |-- prepare-pull-request/
|   `-- update-architecture/
|-- .github/
|   |-- ISSUE_TEMPLATE/
|   |-- pull_request_template.md
|   |-- CODEOWNERS
|   `-- workflows/
|-- apps/web/
|   |-- AGENTS.md
|   `-- src/
|       |-- app/
|       |-- components/
|       |-- features/
|       `-- server/
|           |-- workflows/
|           |-- integrations/AGENTS.md
|           |-- security/
|           `-- observability/
|-- packages/
|   |-- contracts/
|   |-- database/
|   |-- design-system/
|   |-- test-utils/
|   `-- typescript-config/
|-- supabase/
|   |-- AGENTS.md
|   |-- migrations/
|   |-- seed.sql
|   `-- tests/database/
|-- tests/{contract,integration,e2e}/
|-- docs/{architecture,decisions,runbooks,product}/
`-- scripts/{verify,check-migrations,check-env}.mjs
```

ROOT	WEB	PROVIDERS	DATABASE
/AGENTS.md	**apps/web/AGENTS.md**	**integrations/AGENTS.md**	**supabase/AGENTS.md**
Product boundary, commands, tenant safety, jobs, approvals, definition of done, review rules.	Server/client boundaries, voice-text parity, feature slices, UI states, design tokens.	Adapter contract, retry, timeout, webhook auth, redaction, normalization, fixtures.	Forward migrations, RLS in same change, indexes, pgTAP, rollout and rollback.

Focused repository skills

.AGENTS/SKILLS

`add-feature-slice`	Ship one capability with UI, server logic, contracts, states, tests, and docs.	`add-supabase-migration`	Produce migration, indexes, types, pgTAP tests, rollout and rollback notes.
`add-provider-adapter`	Normalize a provider behind fixtures, timeout, retry, idempotency, and redaction.	`change-ai-prompt`	Version prompts and schemas, add eval cases, and state behavior and cost impact.
`diagnose-job-failure`	Build a timeline, root cause, affected set, safe retry path, and prevention test.	`prepare-pull-request`	Collect checks, screenshots, migrations, risks, rollout, and rollback evidence.
`update-architecture`	Refresh diagrams, decisions, guidance, skills, and official-source evidence.	`skill contract`	One recognizable job, clear trigger, explicit input/output, narrow stop conditions.

Codex behavior: repository skills are discovered from `.agents/skills`. Root-to-working-directory AGENTS.md guidance is combined, and closer instructions take precedence. Keep the total instruction chain concise.

LOOP ENGINEERING IN PRACTICE

Small pull requests. Hard checks. Human merge.

The loop gives coding agents a repeatable route, but CI supplies deterministic evidence and a human remains accountable for approval and release.

01	02	03	04	05	06	07	08	09	10
Small issue	Read guidance + skill	Plan acceptance	Branch	Implement	Local checks	Open PR + preview	CI + Codex review	Human approval	Merge + observe

PHASE 0	PHASE 1	PHASE 2	PHASE 3	PHASE 4	PHASE 5	PHASE 6	PHASE 7
Foundation	**SaaS shell**	**Voice cockpit**	**Owned channel**	**Research**	**Ideas**	**Packages**	**Hardening**
Git, pnpm, Next.js, local Supabase, CI, AGENTS.md, skills, decisions.	Auth, workspaces, roles, RLS tests, private buckets, usage ledger.	Realtime speech, transcript review, text fallback, messages, metering.	OAuth, encrypted refresh token, sync, metrics, quota, deletion.	Jobs, Apify, Firecrawl, webhooks, evidence, retries, cancellation.	Patterns, candidates, scoring, evidence explanations, approval.	Titles, thumbnails, hooks, outline, script, versions, validation, export.	Billing, limits, admin, monitoring, retention, recovery, audit readiness.

Cross-tenant leak

A guessed ID must never reveal another workspace.

Control: RLS + negative pgTAP tests

Credential leak

OAuth and service secrets must never reach client code or logs.

Control: private schema + Vault + secret scan

Duplicate callbacks

Apify or other providers may deliver the same event again.

Control: unique receipt + idempotent handler

Runaway spend

Voice, scraping, and model work can outpace customer value.

Control: budgets + append-only usage ledger

Approval bypass

Edited or regenerated output must not inherit an old approval.

Control: immutable versions + state-machine tests

Lost progress

Realtime disconnects must not erase the underlying workflow.

Control: jobs/events are durable truth

Unsupported claims

Generated recommendations must show why they were made.

Control: evidence items + source evals

YouTube quota

Public SaaS demand can exhaust project quota.

Control: cache + accounting + backoff + alerts

Scraper drift

External structures and provider behavior will change.

Control: adapters + fixtures + partial results

Recovery gap

Database backups alone do not contain Storage objects.

Control: database + object recovery exercise

CI contract: format, lint, TypeScript, build, unit tests, contract tests, migration validation, pgTAP RLS tests, Playwright critical paths, secret scan, security checks, preview evidence, human approval.